Version 1.4

September 2026

STRUKTUR DATA

MODUL 3 - STACK and QUEUE

Penulis:

FERY PRATAMA PUTRA

MUHAMMAD IBRAHIM AL AYUBI

MARENO ALI IBRAHIM

INFORMATICS LABORATORY

University of Muhammadiyah Malang

DAFTAR ISI

DAFTAR ISI **2**
PENDAHULUAN **3**
TUJUAN 3
TARGET MODUL 3
PERSIAPAN 3
KEYWORDS 3
REFERENSI **3**
MATERI **4**
1. Stack **4**
1.1 Definisi Stack 4
1.3 Operasi Dasar Stack 5
1.4 Implementasi Stack 6
2. Queue **8**
2.1 Definisi Queue 8
2.2 Karakteristik Queue 8
2.3 Basic Queue Operations 9
2.4 Implementasi Queue 9
CODELAB **13**
Exercise 13
DEMO **16**
Task 1 Food Order Processing System (Queue & Stack) 16
Penilaian **18**
Rubric Penilaian 18
Grading Scale 18

PENDAHULUAN

TUJUAN

1. Mahasiswa mampu memahami dan menjelaskan konsep dasar Java Stack dan Queue.

TARGET MODUL

1. Mahasiswa mampu memahami dan menerapkan konsep stack dan queue.

PERSIAPAN

1. Bagi mahasiswa Muslim, dapat membaca doa berikut sebelum memulai pembelajaran.

رَضِيْتُ بِاللهِ رَبًّا وَبِالإِسْلَامِ دِيْنًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُوْلًا. رَبِّ زِدْنِيْ عِلْمًا وَارْزُقْنِيْ فَهْمًا

2. Laptop atau komputer
3. Java Development Kit
4. Java Runtime Environment
5. IDE (Intellij IDEA, Eclipse, Netbeans, etc.)
6. Motivasi yang kuat dan komitmen untuk belajar

KEYWORDS

Stack, LinkedList, Queue

REFERENSI

https://www.scaler.com/topics/java/stack-and-queue-in-java/.

https://www.educative.io/blog/data-structures-stack-queue-java-tutorial.

https://www.geeksforgeeks.org/queue-using-stacks/.

https://www.geeksforgeeks.org/java/java-program-to-implement-shunting-yard-algorithm/.

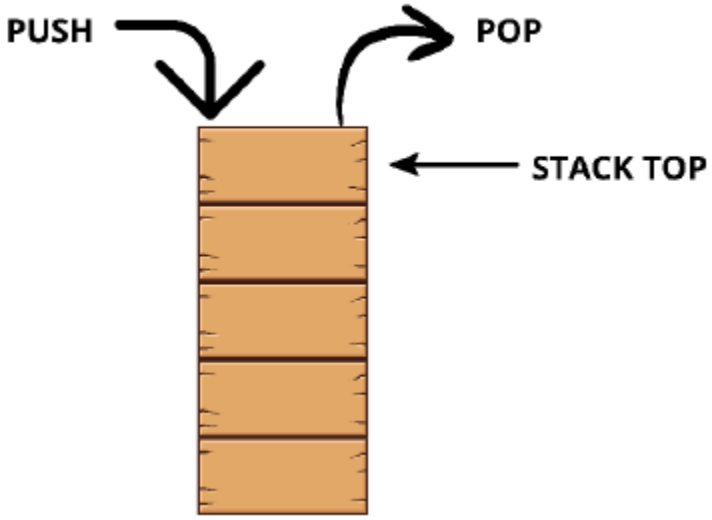
Note: Referensi-referensi tersebut mungkin memiliki sedikit perbedaan dalam penjelasan atau contohnya. Fokuslah pada pemahaman konsep dasar dan penerapannya pada studi kasus yang diberikan dalam modul ini.

MATERI

1. Stack

1.1 Definisi Stack

Stack dapat dianalogikan sebagai tumpukan objek, di mana setiap data diletakkan di atas data lainnya. Setiap elemen dalam Stack hanya dapat ditambahkan atau dihapus dari posisi paling atas. Data yang berada di bagian tengah atau bagian bawah Stack tidak dapat diakses secara langsung. Untuk mencapai elemen-elemen tersebut, seluruh data di atasnya harus dihapus terlebih dahulu. Struktur ini mengikuti aturan akses yang ketat, sehingga stack cocok digunakan dalam situasi ketika data yang paling baru ditambahkan harus diproses terlebih dahulu.



Gambar 1. *Ilustrasi Stack*

1.2 Karakteristik Stack

Stack memiliki pola akses yang sangat terbatas. Seluruh operasi dilakukan pada bagian Top Stack dan tidak diperbolehkan melakukan akses langsung ke elemen di bawahnya. Pembatasan ini memastikan data diproses dalam urutan yang terstruktur.

Stack mengikuti prinsip LIFO (Last-In, First-Out), yang berarti elemen data yang terakhir dimasukkan ke dalam Stack akan menjadi elemen pertama yang dihapus. Karakteristik ini membuat Stack sangat berguna untuk mengelola data sementara, seperti riwayat eksekusi atau operasi undo.

Karakteristik	Deskripsi
LIFO (Last In, First Out)	Data yang terakhir dimasukkan ke dalam stack akan menjadi data pertama yang dihapus. Dalam sistem tiket, tiket terakhir yang dipesan akan menjadi tiket pertama yang dibatalkan.
One-Way Access	Stack hanya memiliki satu titik akses, yaitu bagian atas (TOP). Data tidak dapat diambil dari bagian tengah atau bagian bawah Stack.
No Random Access	Elemen dalam Stack tidak dapat diakses secara langsung menggunakan indeks seperti pada Array atau List.
Ordered	Urutan data ditentukan berdasarkan waktu data tersebut dimasukkan ke dalam Stack.
Suitable for History	Stack cocok digunakan untuk menyimpan riwayat pesanan, pembatalan tiket, serta fitur undo.

1.3 Operasi Dasar Stack

Semua operasi pada Stack dilakukan pada bagian TOP. Terdapat tiga operasi utama yang menentukan cara kerja sebuah Stack..

Method	Deskripsi
`push(data)`	Menambahkan data ke bagian paling atas stack. Contoh: menyimpan riwayat pemesanan tiket baru.
`pop()`	Menghapus dan mengembalikan data yang berada di bagian paling atas Stack. Contoh: membatalkan reservasi tiket terakhir.
`peek()`	Melihat data yang berada di bagian paling atas Stack tanpa menghapusnya. Contoh: memeriksa tiket yang terakhir kali dipesan.
`isEmpty()`	Memeriksa apakah stack kosong atau tidak. Digunakan sebelum menjalankan pop() untuk menghindari error.
`isFull()`	Memeriksa apakah stack sudah penuh, khususnya pada stack berbasis array.

1.4 Implementasi Stack

Using a Stack Library

```java
package com.test;

// File: StackLibraryExample.java
import java.util.Stack;

public class StackLibraryExample {
    public static void main(String[] args) {
        Stack<String> riwayatTiket = new Stack<>();

        // Push ()
        riwayatTiket.push("Tiket KA-001");
        riwayatTiket.push("Tiket KA-002");
        riwayatTiket.push("Tiket KA-003");

        System.out.println("Riwayat Pemesanan: " + riwayatTiket);

        // Peek
        System.out.println("Tiket Terakhir: " + riwayatTiket.peek());

        // Pop ()
        System.out.println("Dibatalkan: " + riwayatTiket.pop());

        System.out.println("Sisa Riwayat: " + riwayatTiket);
    }
}

```

Output

```
❯ /usr/bin/env /usr/lib/jvm/java-17-temurin/bin/java -XX:+Sho
202T110254Z-1-001/Modul/1/modul1/target/classes com.test.Stack
Riwayat Pemesanan: [Tiket KA-001, Tiket KA-002, Tiket KA-003]
Tiket Terakhir: Tiket KA-003
Dibatalkan: Tiket KA-003
Sisa Riwayat: [Tiket KA-001, Tiket KA-002]
```

Menggunakan stack tanpa library

```java
class StackManual {
    private String[] data;
    private int top;
    private int kapasitas;

    public StackManual(int kapasitas) {
        this.kapasitas = kapasitas;
        data = new String[kapasitas];
        top = -1;
    }

    // Push
    public void push(String tiket) {
        if (top == kapasitas - 1) {
            System.out.println("Stack penuh!");
            return;
        }
        data[++top] = tiket;
        System.out.println("Pesan tiket: " + tiket);
    }

    // Pop
    public void pop() {
        if (top == -1) {
            System.out.println("Stack kosong!");
            return;
        }
        System.out.println("Batalkan tiket: " + data[top--]);
    }

    // Tampilkan Stack
    public void tampilkan() {
        System.out.print("Riwayat: ");
        for (int i = 0; i <= top; i++) {
            System.out.print(data[i] + " ");
        }
        System.out.println();
    }
}

public class StackManualExample {
    public static void main(String[] args) {
        StackManual stack = new StackManual(5);

        stack.push("Tiket KA-001");
        stack.push("Tiket KA-002");
        stack.push("Tiket KA-003");

        stack.tampilkan();

        stack.pop();
        stack.tampilkan();
    }
}

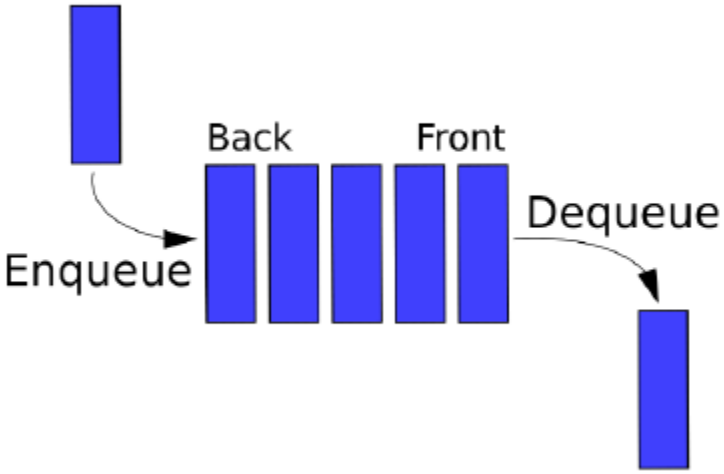
```

2. Queue

2.1 Definisi Queue

Queue adalah kumpulan data di mana elemen hanya dapat ditambahkan dari satu sisi, yang disebut **rear (tail)**, dan elemen hanya dapat dihapus dari sisi yang berlawanan, yang disebut **front (head)**. Struktur ini membatasi cara data diakses, sehingga elemen tidak dapat ditambahkan atau dihapus secara bebas dari posisi apa pun.

Queue bekerja menggunakan dua ujung yang telah ditentukan dengan jelas, yaitu **head**, tempat elemen dihapus, dan **tail**, tempat elemen ditambahkan. Berbeda dengan Stack yang hanya memiliki satu titik akses, Queue memiliki dua titik akses. Queue mengikuti prinsip FIFO **(First-In, First-Out)**, yang berarti elemen data yang pertama kali dimasukkan ke dalam Queue akan menjadi elemen pertama yang dihapus.



Gambar 2. *Ilustrasi Queue*

Dalam kehidupan sehari-hari, queue dapat dianalogikan sebagai antrean pembelian tiket di stasiun kereta api. Penumpang yang datang lebih awal akan dilayani terlebih dahulu. Jika penumpang baru datang untuk membeli tiket, mereka akan ditempatkan di bagian paling belakang antrean. Penumpang yang berada di posisi terakhir akan dilayani paling akhir dan, dalam beberapa situasi, berisiko tidak mendapatkan tiket jika persediaan tiket sudah habis.

2.2 Karakteristik Queue

Queue memiliki karakteristik utama berupa aliran data yang teratur dan adil. Penambahan data hanya dilakukan pada bagian rear (tail), sedangkan penghapusan data hanya dilakukan pada bagian front (head). Elemen yang berada di tengah Queue tidak dapat diakses atau dimodifikasi secara langsung.

Karakteristik tersebut memastikan bahwa urutan pelayanan selalu mengikuti urutan kedatangan. Oleh karena itu, queue sangat cocok digunakan untuk sistem yang

berorientasi pada pelayanan (service-oriented systems), seperti sistem antrean pemesanan tiket, di mana setiap penumpang harus dilayani berdasarkan urutan kedatangannya dalam sistem.

2.3 Basic Queue Operations

Berikut adalah beberapa method penting pada Queue Interface yang sering digunakan dalam implementasi menggunakan library Java. Tabel ini membantu praktikan memahami fungsi masing-masing method secara konseptual sebelum mulai menulis kode.

Method	Deskripsi
`add(element)`	Menambahkan elemen ke bagian belakang (tail) Queue. Jika Queue sudah penuh, method ini akan menghasilkan exception.
`offer(element)`	Menambahkan elemen ke dalam Queue. Lebih aman daripada add() karena tidak menghasilkan exception ketika Queue penuh, melainkan mengembalikan nilai false.
`peek()`	Melihat elemen yang berada di bagian depan Queue tanpa menghapusnya. Mengembalikan null jika Queue kosong.
`element()`	Mirip dengan peek(), tetapi akan menghasilkan NoSuchElementException jika Queue kosong.
`poll()`	Menghapus dan mengembalikan elemen di bagian depan Queue. Mengembalikan null jika Queue kosong.
`remove()`	Menghapus dan mengembalikan elemen di bagian depan Queue. Akan menghasilkan exception jika Queue kosong.

2.4 Implementasi Queue

Implementasi Queue dapat dilakukan dengan dua pendekatan utama, yaitu menggunakan library bawaan Java atau membangun struktur Queue secara manual tanpa bergantung pada dukungan library.

Pada implementasi berbasis library, Java menyediakan Queue Interface yang umumnya diimplementasikan menggunakan LinkedList. Pendekatan ini menyederhanakan pengelolaan Queue sehingga programmer dapat berfokus pada logika aplikasi tanpa harus menangani detail internal struktur data.

Pada implementasi manual, Queue dapat dibangun menggunakan Linked List dengan bantuan sebuah Class Node.

Struktur ini menggunakan dua referensi utama, yaitu head dan tail, untuk mengelola operasi enqueue dan dequeue. Pendekatan manual sangat penting untuk

membantu mahasiswa memahami cara kerja Queue secara mendasar dan menyadari bahwa Queue pada dasarnya merupakan aliran data yang terkontrol berdasarkan aturan FIFO.

1. **Implementasi queue menggunakan library Java**

```java
import java.util.LinkedList;
import java.util.Queue;

public class QueueLibraryExample {
    public static void main(String[] args) {
        Queue<String> ticketQueue = new LinkedList<>();

        ticketQueue.add("Passenger A");
        ticketQueue.add("Passenger B");
        ticketQueue.add("Passenger C");

        System.out.println("Queue: " + ticketQueue);
        System.out.println("Served: " + ticketQueue.poll());
        System.out.println("Remaining Queue: " + ticketQueue);
    }
}
```

Output

```
❯ /usr/bin/env /usr/lib/jvm/java-17-temurin/bin
202T110254Z-1-001/Modul/1/modul1/target/classes
Queue: [Passenger A, Passenger B, Passenger C]
Served: Passenger A
Remaining Queue: [Passenger B, Passenger C]
```

2. Implementasi queue tanpa menggunakan library Java

1. Membuat Class Node

```java
class Node {
    String data;
    Node next;

    Node(String data) {
        this.data = data;
        this.next = null;
    }
}

```

2. Membuat Class QueueList

```java
class QueueList {
    private Node head, tail;

    public QueueList() {
        head = tail = null;
    }

    // Enqueue
    public void enqueue(String data) {
        Node newNode = new Node(data);

        if (tail == null) {
            head = tail = newNode;
            return;
        }
        tail.next = newNode;
        tail = newNode;
    }

    // Dequeue
    public void dequeue() {
        if (head == null) {
            System.out.println("Queue is empty!");
            return;
        }
        System.out.println("Served: " + head.data);
        head = head.next;

        if (head == null) {
            tail = null;
        }
    }

    public void display() {
        Node temp = head;
        System.out.print("Queue: ");
        while (temp != null) {
            System.out.print(temp.data + " ");
            temp = temp.next;
        }
        System.out.println();
    }
}
```

3. Membuat Class Main

```java
public class QueueManualMain {
    public static void main(String[] args) {
        QueueList queue = new QueueList();

        queue.enqueue("Passenger A");
        queue.enqueue("Passenger B");
        queue.enqueue("Passenger C");

        queue.display();
        queue.dequeue();
        queue.display();
    }
}
```

Output

```
❯ cd /home/mipow/Downloads/drive-download-20251
tionMessages -cp /home/mipow/Downloads/drive-dow
Queue: Passenger A Passenger B Passenger C
Served: Passenger A
Queue: Passenger B Passenger C
```

CODELAB

Dalam CODELAB ini, mahasiswa diberikan sebuah program Java lengkap yang mendemonstrasikan penggunaan dasar Queue (FIFO) dan Stack (LIFO) dalam konteks sederhana sistem pemesanan tiket. Mahasiswa diminta untuk melengkapi kode dan memahami bagaimana Queue digunakan untuk mengelola antrean pelanggan yang memesan tiket, sedangkan Stack digunakan untuk menyimpan riwayat transaksi.

Output expected:

```
Initial Ticket Customer Queue:
[Ayub, Reno, Budi]
Serving customer: Ayub
Queue after serving:
[Reno, Budi]
Next customer to serve: Reno

Ticket Transaction History:
[Ticket-Transaction-1, Ticket-Transaction-2, Ticket-Transaction-3]
Undo last transaction: Ticket-Transaction-3
Ticket Transaction History after undo:
[Ticket-Transaction-1, Ticket-Transaction-2]
Current top transaction: Ticket-Transaction-2
Ticket transaction history is not empty. Size: 2
```

Exercise

1. Silakan lengkapi kode implementasi struktur data Stack dan Queue menggunakan library Java dengan jelas dan benar, sehingga kamu dapat memahami output program dan memperdalam pemahaman mengenai konsep tersebut.

```java
package com.test;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class StackQueueCodelab {
    public static void main(String[] args) {

        // ************************************************************
        // BAGIAN 1: DEMONSTRASI QUEUE (FIFO)
        // ************************************************************
        // Queue digunakan untuk mengelola antrean pelanggan tiket.
        // Pelanggan yang datang pertama akan dilayani terlebih dahulu.
        // ************************************************************

        // TODO 1: Inisialisasi Queue untuk antrean pelanggan tiket
        // Petunjuk: Queue adalah sebuah interface,
        // implementasinya dapat menggunakan LinkedList
        // Sintaks: Queue<TipeData> namaVariabel = new LinkedList<>();
        Queue<String> ticketQueue = ____________________;

        // TODO 2: Tambahkan pelanggan ke dalam Queue menggunakan add()
        // Petunjuk: Gunakan method add(element) untuk menambahkan
        // pelanggan ke dalam antrean
        // Sintaks: namaQueue.add("data");

        // Tambahkan tiga pelanggan: Andi, Titan, Rapi
        ticketQueue._________("Andi");

        ticketQueue._________("Titan");

        ticketQueue._________("Rapi");

        // Menampilkan antrean awal
        System.out.println("Antrean Pelanggan Tiket Awal:");
        System.out.println(ticketQueue);

        // TODO 3: Layani pelanggan pertama menggunakan poll()
        // Petunjuk: Gunakan poll() untuk mengambil dan menghapus
        // elemen pertama dari Queue
        // Sintaks: String hasil = namaQueue.poll();
        // poll() mengembalikan elemen sekaligus menghapusnya
        // dari Queue

        String servedCustomer = ticketQueue._________();

        System.out.println("Melayani pelanggan: " + servedCustomer);

        // Menampilkan antrean setelah pelanggan pertama dilayani
        System.out.println("Antrean setelah pelayanan:");
        System.out.println(ticketQueue);

        // TODO 4: Lihat pelanggan berikutnya tanpa menghapusnya
        // menggunakan peek()
        // Petunjuk: Gunakan peek() untuk melihat elemen pertama
        // tanpa menghapusnya
        // Sintaks: String nextCustomer = namaQueue.peek();

        String nextCustomer = ticketQueue._________();

        System.out.println("Pelanggan berikutnya yang akan dilayani: "
                + nextCustomer);
```

```
        // ************************************************************
        // BAGIAN 2: DEMONSTRASI STACK (LIFO)
        // ************************************************************
        // Stack digunakan untuk menyimpan riwayat transaksi tiket.
        // Transaksi yang terakhir ditambahkan akan menjadi transaksi
        // pertama yang dapat dibatalkan.
        // ************************************************************

        // TODO 5: Inisialisasi Stack untuk riwayat transaksi tiket
        // Petunjuk: Gunakan class Stack untuk membuat sebuah Stack
        // Sintaks: Stack<TipeData> namaVariabel = new Stack<>();

        Stack<String> transactionHistory = ____________________;

        // TODO 6: Tambahkan transaksi ke dalam Stack menggunakan push()
        // Petunjuk: Gunakan push(element) untuk menambahkan transaksi
        // ke dalam Stack
        // Sintaks: namaStack.push("data");

        // Tambahkan tiga transaksi tiket
        transactionHistory.__________("Ticket-Transaction-1");

        transactionHistory.__________("Ticket-Transaction-2");

        transactionHistory.__________("Ticket-Transaction-3");

        // Menampilkan riwayat transaksi tiket
        System.out.println();
        System.out.println("Riwayat Transaksi Tiket:");
        System.out.println(transactionHistory);

        // TODO 7: Batalkan transaksi terakhir menggunakan pop()
        // Petunjuk: Gunakan pop() untuk mengambil dan menghapus
        // elemen paling atas dari Stack
        // Sintaks: String lastTransaction = namaStack.pop();

        String lastTransaction = transactionHistory.__________();

        System.out.println("Membatalkan transaksi terakhir: "
                + lastTransaction);

        // Menampilkan riwayat transaksi setelah pembatalan
        System.out.println("Riwayat Transaksi Tiket setelah pembatalan:");
        System.out.println(transactionHistory);

        // TODO 8: Lihat transaksi terakhir tanpa menghapusnya
        // menggunakan peek()
        // Petunjuk: Gunakan peek() untuk melihat elemen paling atas
        // tanpa menghapusnya
        // Sintaks: String topTransaction = namaStack.peek();

        String topTransaction = transactionHistory.__________();

        System.out.println("Transaksi terakhir saat ini: "
                + topTransaction);

        // TODO 9: Periksa apakah riwayat transaksi kosong
        // menggunakan isEmpty()
        // Petunjuk: Gunakan isEmpty() untuk memeriksa apakah Stack kosong
        // Sintaks: boolean kosong = namaStack.isEmpty();

        if (transactionHistory.__________()) {

            System.out.println("Tidak ada riwayat transaksi tiket.");

        } else {

            // TODO: Dapatkan jumlah transaksi menggunakan size()
            System.out.println(
                "Riwayat transaksi tiket tidak kosong. Jumlah: "
                + transactionHistory.__________()
            );
        }
    }
}
```

DEMO

Task 1 Food Order Processing System (Queue & Stack)

Pada modul sebelumnya, Anda telah mengembangkan Food Ordering System yang mengelola beberapa pesanan pelanggan dan item menu menggunakan ArrayList dan LinkedList.

Pada modul ini, Anda akan melanjutkan pengembangan sistem yang sama dengan menambahkan struktur data Queue dan Stack. Queue akan digunakan untuk mengelola urutan pemrosesan pesanan, sedangkan Stack akan digunakan untuk menyimpan riwayat pesanan yang telah diproses serta mendukung operasi pembatalan (undo).

Implementasi

Lanjutkan Food Ordering System dari **modul sebelumnya**, mahasiswa akan melakukan modifikasi terutama pada class berikut:

1. Class OrderManager
   Tambahkan Queue<FoodOrder> untuk mengelola pesanan pelanggan yang sedang menunggu diproses menggunakan konsep FIFO, serta Stack<FoodOrder> untuk menyimpan riwayat pesanan yang telah diproses menggunakan konsep LIFO.
2. Class Main
   Perbarui antarmuka dan menu pengguna agar pengguna dapat menambahkan pesanan, menampilkan antrean pesanan, memproses pesanan berikutnya, membatalkan transaksi terakhir, serta menampilkan riwayat transaksi.

Class Customer, FoodOrder, OrderItem, dan FoodCategory yang sudah ada sebaiknya digunakan kembali dari modul sebelumnya.

Mahasiswa harus mengimplementasikan fitur-fitur berikut:

1. Queue-Based Order Processing (FIFO)
   Gunakan Queue untuk mengelola pesanan pelanggan yang sedang menunggu untuk diproses. Sistem harus mendukung:
   a. Menambahkan pesanan pelanggan yang telah selesai dibuat ke dalam Queue (Create)
   b. Menampilkan antrean pesanan saat ini (Read)
   c. Memproses pesanan berdasarkan urutan First-In-First-Out (FIFO)
   d. Menghapus pesanan yang telah diproses dari daftar pesanan aktif
2. Stack-Based Transaction History (LIFO)
   Gunakan Stack untuk menyimpan pesanan yang telah diproses.
   a. Setiap pesanan yang telah diproses harus dimasukkan (push) ke dalam Stack
   b. Menampilkan riwayat pesanan yang telah diproses
   c. Mendukung pembatalan pesanan yang terakhir diproses menggunakan konsep Last-In-First-Out (LIFO)

d. Ketika sebuah pesanan dibatalkan, kembalikan pesanan tersebut ke daftar pesanan aktif dan antrean pemrosesan

3. Implementasi Class Main.

Output like this:

```
=== Food Ordering System ===
1. Add Customer Order
2. Display Active Orders
3. Display Order Queue
4. Process Next Order
5. Undo Last Transaction
6. Display Transaction History
7. Exit
Choose menu: 1

--- Entering Customer Order ---
Enter Customer Name: ayub
Enter Customer ID: 1
Enter Order Code: cs-01
Enter Menu Name: geprek
Select Food Category:
1. FOOD
2. BEVERAGE
3. DESSERT
Enter choice: 1
Enter Quantity: 1
Add another item to this order? (y/n): n

Order added successfully.
Order cs-01 is waiting in the processing queue.
```

```
=== Food Ordering System ===
1. Add Customer Order
2. Display Active Orders
3. Display Order Queue
4. Process Next Order
5. Undo Last Transaction
6. Display Transaction History
7. Exit
Choose menu: 2

==================================
=== ALL ACTIVE ORDERS ===

==================================
Order Code       : cs-01
Customer Name    : ayub
Customer ID      : 1
Customer ID Type : Long
Ordered Items:
  - geprek x1 (FOOD)
```

```
=== Food Ordering System ===
1. Add Customer Order
2. Display Active Orders
3. Display Order Queue
4. Process Next Order
5. Undo Last Transaction
6. Display Transaction History
7. Exit
Choose menu: 3

==================================
=== ORDER PROCESSING QUEUE ===
1. cs-01 - ayub
```

```
=== Food Ordering System ===
1. Add Customer Order
2. Display Active Orders
3. Display Order Queue
4. Process Next Order
5. Undo Last Transaction
6. Display Transaction History
7. Exit
Choose menu: 4

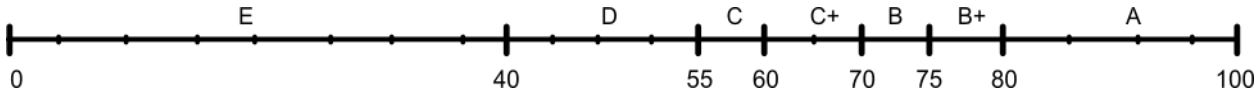
==================================
=== PROCESSING ORDER ===
Processing order: cs-01
Customer: ayub
Order processed successfully.
```

Penilaian

Rubric Penilaian

Assessment Aspect	Points (Total 100%)
Codelab	**Total 20%**
Successfully run smoothly	20%
Demo	**Total 80%**
Running smoothly: Task 1	10%
Implement Stack operations	20%
Implement Queue operations	20%
Answer Questions	30%

Grading Scale



A = (81 - 100) → Sepuh

B+ = (75 - 80) → Very Good

B = (70 - 74) → Good

C+ = (60 - 69) → Fairly Good

C = (55 - 59) → Fair

D = (41 - 54) → Poor

E = (0 - 40) → Bro really...